

Metodo delle Correnti di Maglia

Risoluzione automatica di circuiti elettrici

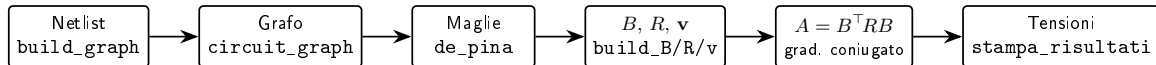
Programmazione e Calcolo Scientifico

2026

Obiettivo e pipeline del programma

Input: una *netlist* (resistori e generatori ideali di tensione).

Output: tensione e corrente su ciascun resistore.



- Si costruisce il grafo del circuito, si trovano le maglie (cicli fondamentali).
- Si assemblano incidenza B , resistenze R , termini noti $\mathbf{v}$ e si risolve $B^T R B \mathbf{i} = \mathbf{v}$.
- Dalle correnti di maglia $\mathbf{i}$ si ricavano le tensioni: $\mathbf{v}_R = R B \mathbf{i}$.

Il circuito come grafo non orientato

`undirected_edge`: lato con nodi sempre ordinati ($\text{from} < \text{to}$)

Un'unica scelta, tre vantaggi:

- (u, v) e (v, u) sono lo stesso lato $\Rightarrow$ niente duplicati;
- il lato è usabile come chiave in `map/set` (ordine lessicografico);
- fissa un *verso canonico* dal nodo minore al maggiore: ogni lato ha così un orientamento di riferimento fisso, rispetto al quale assegnare senza ambiguità il segno (± 1) con cui ciascuna maglia lo attraversa in B .

`undirected_graph`: due strutture in parallelo

- mappa di adiacenza `map<T, set<T>>` $\rightarrow$ per le visite (DFS, BFS, Dijkstra);
- vettore di lati *numerati* `vector<undirected_edge>` $\rightarrow$ numerazione stabile per indicizzare righe/colonne di B , R e i vettori di De Pina.

Tenere entrambe evita ricostruzioni continue, al costo di una ridondanza $O(n + m)$.

Separazione tra grafo e parte elettrica

- `circuit_graph` usa la **composizione**: incapsula un `undirected_graph` (la topologia) e gli associa i `component` (i dati elettrici: tipo, valore, polarità).
- Resistori e generatori sono tenuti in **due liste separate**:
 - i resistori alimentano le matrici B ed R ;
 - i generatori alimentano il termine noto $\mathbf{v}$.
- `eigen_support` **confina** tutta l'algebra lineare (prodotti, gradiente coniugato, numero di condizionamento) sopra la libreria Eigen, in un solo punto.

Perché

Responsabilità separate: il livello del grafo e quello elettrico evolvono in modo indipendente, così si può modificare il modello elettrico senza toccare gli algoritmi sui grafi (e viceversa) e ogni parte resta semplice da sviluppare e testare in isolamento.

Le maglie: due metodi per i cicli fondamentali

Un grafo connesso ha esattamente $k = m - n + 1$ cicli fondamentali. Come richiesto dalla consegna si sono implementati i seguenti metodi per individuarli.

`find_cycles` - DFS + coalbero

Albero di supporto con DFS, coalbero $C = G - T$; ogni lato del coalbero, reinserito, chiude una maglia. Semplice e veloce, ma i cicli *non* sono necessariamente minimi. Selezionabile a runtime con l'opzione `--no-minimi`.

`de_pina` - base di cicli minimi (*metodo di default*)

Per ogni vettore di base S_i cerca il ciclo di peso minimo con *intersezione dispari* con S_i . Usa un grafo “liftato” (segnato): ogni nodo è sdoppiato in (nodo, +) e (nodo, -) tramite `LiftingNode`, e si calcolano cammini minimi su di esso. Più costoso, ma produce maglie minime e più naturali da interpretare.

Dai cicli al sistema lineare

Dalle maglie si assemblano le tre strutture:

- **Incidenza** B (build_B): $B_{ij} \in \{+1, -1, 0\}$ secondo il verso con cui la maglia j attraversa il resistore i (segni dati da plus_minus);
- **Resistenze** R (build_R): matrice diagonale con i valori dei resistori;
- **Termine noto** $\mathbf{v}$ (build_v): somma dei generatori, segni dati da gen_sign.

$$\underbrace{B^T R B}_A \mathbf{i} = \mathbf{v} \quad \Longrightarrow \quad \mathbf{v}_R = R B \mathbf{i}$$

Risoluzione: gradiente coniugato

$A = B^T R B$ è **simmetrica e definita positiva** quando R è diagonale positiva e B ha rango pieno (garantito da entrambi i metodi). Questo giustifica il gradiente_coniugato.

Problema dei segni (1): i resistori

La **difficoltà principale** incontrata è stata tenere conto correttamente di tutti i segni e dei versi di percorrenza sul grafo.

`plus_minus(resistore, ciclo)`

Si confronta il verso della maglia con il verso canonico del lato (minore→maggiore):

+1 se la maglia lo percorre $\text{min} \rightarrow \text{max}$, -1 se $\text{max} \rightarrow \text{min}$, 0 se assente.

L'ordinamento canonico del lato rende questo confronto *ben definito* e senza ambiguità.

Problema dei segni (2): i generatori e i versi

`gen_sign(generator, ciclo)` - convenzione *opposta* ai resistori

+1 se la maglia attraversa il generatore da “-” a “+”, -1 da “+” a “-”.

Per le resistenze il segno è positivo entrando dal “+”; per i generatori è il contrario.

- Il verso di una maglia è una scelta arbitraria: invertirlo cambia segno alla sua colonna di B , al termine noto e quindi alla corrente I_j , ma le tensioni $\mathbf{v}_R = R B \mathbf{i}$ restano **invariate** (i due segni si annullano), verificato sui circuiti d'esempio della consegna.

Come l'abbiamo gestito

Tutta la logica dei segni è *isolata* in due sole funzioni piccole (`plus_minus`, `gen_sign`), testate separatamente: così un eventuale errore di segno si individua e si corregge in un unico punto, senza propagarsi nel resto del codice.

Costi (1): primitive su grafo e ricerca dei cicli

Notazione: n nodi, m lati, $k = m - n + 1$ maglie.

Operazione	Costo	Motivo
<code>add_edge</code>	$O(\log n)$	inserimenti in mappa e insiemi
<code>all_edges()</code>	$O(m \log m)$	ricostruisce un set dal vettore
<code>edge_number(e)</code>	$O(m)$	scansione lineare del vettore
<code>operator- (G - T)</code>	$O(m^2 \log m)$	ricostruisce <code>all_edges()</code> ogni iter.
<code>recursive_dfs</code>	$O((n+m) \log n)$	ogni nodo visitato una volta
<code>dijkstra</code>	$O((n+m) \log n)$	coda con <code>std::set</code> , pesi unitari

- DFS + coalbero: complessivamente $\sim O(n^2 \log n)$.
- De Pina: $\sim O(k \cdot n^2 \log n) \approx O(n^3 \log n)$ per $m, k = O(n)$.
- **Il collo di bottiglia è la ricerca dei cicli**, non l'algebra lineare.

Costi (2): sistema lineare e punti di forza

Assemblaggio e risoluzione

Fase	Costo
build_B	$O(m_R k L)$
build_v	$O(k m_V L)$
$A = B^\top R B$	$O(k^2 m_R)$
grad. coniugato	$O(k^3)$

Algebra lineare complessiva $\sim O(n^3)$: confrontabile con DFS, inferiore a De Pina.

Punti di forza

- dipendenza da Eigen *isolata* in un solo modulo;
- due metodi di ricerca cicli intercambiabili;
- numerazione stabile dei lati per indicizzare le matrici;
- separazione topologia/parte elettrica $\rightarrow$ riuso e testabilità.

Costi (3): punti di debolezza (Dijkstra a pesi unitari)

Il problema

Dentro De Pina i cammini minimi sul grafo liftato sono calcolati con dijkstra, ma i **pesi sono tutti**

1. Dijkstra paga allora un fattore $O(\log n)$ in più (coda di priorità con `std::set`) del tutto inutile.

- Una **BFS** risolve lo stesso problema in $O(n + m)$, senza il fattore logaritmico.
- I cammini si ricalcolano *per ogni nodo $\times$ ogni vettore di base*: quel $\log n$ si paga molte volte $\Rightarrow$ sostituendo Dijkstra con BFS si velocizza parecchio.

Lo abbiamo testato

Abbiamo implementato `bfs()` come sostituto immediato (restituisce lo stesso `DijkstraResult`, riusando `get_path_vec`) ed esposto i flag `--bfs` e `--time` nel main. Misurando con il `Timer`, sui circuiti più grandi la BFS è **costantemente più veloce** di Dijkstra.

I test costruiti e perché

Quattro test isolano un singolo **strato** della pipeline; un quinto **confronta i due metodi**:

- **test_grafo**: primitive del grafo: lato duplicato ignorato, albero/coalbero con $|E| - |V| + 1$, cammino di Dijkstra. *È la base su cui poggia tutto il resto.*
- **test_matrici**: nucleo numerico: gradiente coniugato su un sistema 2×2 SPD noto ($x = \frac{1}{11}, \frac{7}{11}$) e numero di condizionamento. *Verifica la convergenza alla soluzione analitica.*
- **test_parser**: robustezza: netlist “sporca” con righe vuote, soli spazi e righe malformate.
- **test_dominio**: plus_minus e gen_sign. *Colpisce direttamente la difficoltà principale: i segni.*
- **test_confronto**: esegue find_cycles e de_pina sullo stesso circuito e verifica che le correnti sui rami coincidano (tol 10^{-9}). *Il risultato fisico non dipende dalla base di cicli scelta.*

Perché questi test

Ogni test isola uno strato: se qualcosa si rompe, il test che fallisce indica subito *dove* intervenire. Ci siamo concentrati sui punti più a rischio, i segni (la difficoltà principale) e la robustezza del parser, validando ogni mattone prima di comporli nel risultato finale.